

———— // ZPWR-CLIP-ENGINE // ————

zpwr-clip-engine

v0.1.0

REFERENCE MANUAL

one arranger, every app, one grid – content swapped, never re-implemented

2026-07-13

MenkeTechnologies

———— MENKETECHNOLOGIES ————

Overview

Shared component. "One arranger, every app, one grid – content swapped, never re-implemented." The sequencer/arranger frontend and its scheduling engine were extracted verbatim from `zpw-r-daw`; hosts mount the grid through a single entry point, `initClipSeq({ nf, getCAT })`, and supply a backend.

`zpw-clip-engine` is the shared DAW arranger / sequencer for the MenkeTechnologies GUI app stack. It is one canvas-2D grid with swappable domains – arranger, notes/piano-roll, launcher/session, automation lanes – plus the clip palette, MIDI export, and project/bank serialization. The real scheduling engine lives here too (`engine/`): the pattern → events → step/transport/MIDI scheduler, extracted from `zpw-r-daw` as a pure-C++17, header-only C ABI (`zpc_clip_*`) – header-only `ClipEngine` + `MidiFile`, no JUCE, no external deps. Created by MenkeTechnologies.

One grid, swappable domains

A single canvas grid renders arranger, piano-roll, launcher/session, and automation-lane domains – content is swapped, never re-implemented per app.

One engine, every consumer

The `zpc_clip_*` C ABI builds two ways from one source: `libzpw-clip-engine-static.a` (the daw links it natively) and `libzpw-clip-engine.dylib/.so` (the Tauri apps load it through a Rust FFI wrapper).

Mount through one entry

Hosts call `initClipSeq({ nf, getCAT })` and supply a backend; the `nf` functions bridge the grid to whichever engine the host uses.

No JUCE, no deps

The engine is header-only C++17 (`ClipEngine` + `MidiFile`) with no JUCE and no external dependencies – so it embeds in native and Tauri hosts alike.

Audio & video clips

Beyond MIDI patterns, tracks can hold audio clips (waveform + fades) and video clips (filmstrip + in/out trim). Video clips drop in-browser (frames sampled to a filmstrip), and a program monitor – dockable or popped out into a draggable, resizable player – previews the frame under the playhead. The foundation for a video-editing timeline.

Hosts

- `zpwir-daw` (JUCE / C++) – drives the grid through native `nf` functions backed by this engine plus its own audio voices / per-track instruments (real sound).
- Tauri apps (Audio-Maxor, traderview, ztranslator) – bridge `nf` → the C ABI via a Rust FFI wrapper.
- Domains – arranger, notes / piano-roll, launcher / session, automation lanes.
- Clip palette – the swappable clip bank shared across domains.
- MIDI export – pattern/event data written out via the engine's `MidiFile`.
- Serialization – project / bank save & load.

Mounting

A host mounts the shared grid through one entry point and supplies its backend bridge:

```
import { initClipSeq } from './webui/clip/clip-seq.js';

// nf = the host's bridge functions (native in the daw, FFI in Tauri apps);
// getCAT = the host's content/category provider for the clip palette
initClipSeq({ nf, getCAT });
```

Layout

See the [engineering report](#) for the module map across [webui/](#) (the shared frontend) and [engine/](#) (the C-ABI scheduler).

MenkeTechnologies · [zpwir-clip-engine](#) · shared arranger / sequencer + clip engine · [zpwir-clip-engine](#) · [MenkeTechnologiesMeta](#)

getCAT() contract

`clip-seq.js` reads exactly four fields off the catalog; the grid domains never touch it:

field	basic backend	meaning
<code>hasClipSeq</code>	<code>true</code>	a step sequencer exists (drives the clip-view clock)
<code>hasClipAudio</code>	<code>false</code>	native JUCE audio-clip engine wired (daw only)
<code>hasKeyboard</code>	<code>true</code>	playback is allowed (gate on <code>clipStart</code>)
<code>name</code>	<code>'clip'</code>	host id; the daw uses <code>'zpwr-daw'</code> (per-track instruments)

nf(name) contract

`nf(name)` returns a per-command function. `clip-seq.js` wraps almost every call in `try/catch` and persists pattern/arrangement/project state itself via `localStorage` (`zfx_clip_*`, `zfx_clip_arrangement`, ...), so most commands can be neutral resolves. The basic backend never throws and produces no audio.

command	daw (native)	basic backend
<code>clipSeqPattern</code>	loads the pattern into the audio sequencer	no-op resolve (UI persists to <code>localStorage</code>)
<code>clipSeqTransport</code>	sets steps/bpm/div/swing/loop on the engine	real – configures the JS step timer
<code>clipSeqPlay</code>	starts/stops the audio-thread clock	real – starts/stops a <code>setInterval</code> step timer
<code>clipSeqStep</code>	returns the audio clock's current step	real – returns the JS timer's current step
<code>clipSeqExportMidi</code>	writes the SMF via native file IO	real – decodes base64 SMF, browser download
<code>clipSeqSaveFile</code>	native file save (only when <code>window.Juce</code>)	no-op resolve
<code>clipAudioLoad</code>	loads an audio sample, returns peaks/dur	no-op resolve <code>null</code> (never reached)
<code>clipAudioTrigger</code>	fires a native audio voice	no-op resolve (never reached)
<code>clipAudioStopAll</code>	stops all audio voices	no-op resolve (never reached)
<code>setXfade</code>	sets the audio-clip crossfade	no-op resolve
<code>loadTrackInstrument</code>	loads a hosted VST3/AU into a track	no-op resolve
<code>dawSetTrackInstrument</code>	turns a track into a built-in voice synth (empty patch = default note-pitched voice)	no-op resolve
<code>dawClearTrackInstrument</code>	removes a track's built-in voice	no-op resolve
<code>trackPresetSave</code>	writes the active track's graphs to a named <code>.ztrk</code> (<code>{type,note?,stereo?}</code>)	no-op resolve
<code>trackPresetList</code>	lists the saved <code>.ztrk</code> presets (<code>[{name,type}]</code>)	no-op resolve <code>[]</code>
<code>trackPresetLoad</code>	reads a named <code>.ztrk</code> preset back	no-op resolve <code>""</code>
<code>trackPresetDelete</code>	deletes a named <code>.ztrk</code> preset	no-op resolve
<code>getPatch/setPatchJson</code>	reads/writes the track's note graph (preset store)	no-op resolve
<code>trkgetPatch/trksetPatchJson</code>	reads/writes the track's stereo graph (preset store)	no-op resolve
<code>midiNoteOn</code>	native MIDI note-on (audible)	no-op resolve (no synth attached)
<code>midiNoteOff</code>	native MIDI note-off	no-op resolve
<code>setActiveLayer</code>	selects the active synth layer	no-op resolve
<code>setTrackCount</code>	sizes the per-track audio graphs	no-op resolve
<code>clipAutoValue</code>	emits a live automation CC value	no-op resolve

Any command the UI adds later but the table omits falls through to a generic neutral resolver (resolve `undefined`), so the host degrades rather than throws.

Ableton Live Set (.als) import

`grid/als-import.js` reads an Ableton Live Set and returns a `serializeProject()`-shaped object, so the host feeds it straight through `deserializeProject` + `applyProjectState` – the same path as a native `.zdp`. This is the cross-DAW interop Bitwig ships: an Ableton arrangement comes in as tracks, clips, MIDI notes, tempo, time signature, arrangement placement, locators, and audio-clip sample paths. Devices/plugins, automation envelopes and audio warping have no target in the clip model and are dropped (as in any cross-DAW import).

`.als` is gzip XML; the host's `gunzipText` decompresses, then routes text starting with `<Ableton` to `alsToProject`. The parser is a dependency-free XML walker (no `DOMParser`, which is absent under Node) so it runs headless in CI. Verified against real sets spanning Live 8.2.1 → Live 12.2.

Float time. A note's exact start is encoded in its fractional grid-step key ("`12.5`" = step $12\frac{1}{2}$); its exact length rides in the optional `bl` field (beats). Integer keys + absent `bl` are the legacy step-sequencer shape, unchanged – old `.zdp` projects load byte-identically. The canvas renderer draws notes at the sub-cell offset and the JS clock micro-delays their note-on. The native step engine (`engine/include/zpc/ClipEngine.h`, the single-clip preview path) has no sub-step slot, so it rounds a fractional start to the nearest step; arrangement playback runs on the float-accurate JS clock.

Tests

The grid/domain unit tests live in `webui/grid/tests/` and run with the Node test runner:

```
node --test webui/grid/tests/
```

Architecture

zpw-clip-engine is consumed two ways from one source. The frontend (`webui/`) is mounted by hosts through `initClipSeq({ nf, getCAT })`: one canvas grid renders swappable domains (arranger, notes/piano-roll, launcher/session, automation lanes) plus the clip palette. The engine (`engine/`) is the scheduler as a pure-C++17, header-only C ABI (`zpc_clip_*`) – `ClipEngine` + `MidiFile`, no JUCE, no external deps – built both as a static lib the JUCE daw links natively and as a `dylib/.so` the Tauri apps load through a Rust FFI wrapper. The `nf` bridge is what differs per host; the grid and the scheduler do not.

Module map

area	responsibility
<code>webui/grid</code>	The canvas grid – model, interactions, cursors, sequencer, project state, domain entry (<code>grid-core.js</code> , <code>grid-model.js</code> , <code>grid-interactions.js</code> , <code>sequencer.js</code> , <code>project.js</code> , <code>als-import.js</code>).
<code>webui/clip</code>	The clip layer – clip grid, clip bank/palette, sequencer view (<code>clip-seq.js</code> , the host entry point), and the non-audio basic backend for preview (<code>clip.js</code> , <code>clip-grid.js</code> , <code>clip-bank.js</code> , <code>clip-basic-backend.js</code>).
<code>webui/lib</code>	Shared frontend helpers used across the clip and grid layers.
<code>engine/</code>	The C++17 header-only scheduler exposed as the <code>zpc_clip_*</code> C ABI – <code>ClipEngine</code> (pattern → events → step/transport) and <code>MidiFile</code> (SMF write). Builds as a static lib and a shared lib from one source.
<code>bindings/</code>	The Rust FFI wrapper (<code>bindings/src/lib.rs</code>) the Tauri apps load, bridging <code>nf</code> calls onto the C ABI.